

Graph Centrality Metrics: Closeness Centrality

Efficient Parallel Implementation using C++ and OpenMP

Course Project Report

[Your Name(s)]

May 2026

Abstract

Graph centrality metrics assign a numerical importance score to each vertex (or edge) in a weighted or unweighted graph. This report focuses on *closeness centrality*, one of the most informative node-importance measures in network analysis. We study its mathematical definition, design efficient sequential and parallel algorithms for computing it at scale, and evaluate our implementation on the real-world *Wiki-Vote* dataset (≈ 7115 nodes, ≈ 103689 edges) as well as on synthetic sparse/dense graphs. Our C++ OpenMP implementation achieves up to **4.56** $\times$ speedup over the single-threaded baseline on unweighted graphs and **5.17** $\times$ on weighted graphs with 8 threads. We compare our results against the Python NetworkX baseline, demonstrating substantial performance improvements in wall-clock time.

Contents

1	Introduction	3
1.1	Motivation and Scope	3
2	Problem Definition	3
2.1	Graph Notation	3
2.2	Closeness Centrality — Classical Definition	3
2.3	Wasserman–Faust Normalisation for Disconnected Graphs	4
3	Algorithms	4
3.1	Overview	4
3.2	Breadth-First Search (Unweighted)	4
3.3	Dijkstra’s Algorithm (Weighted)	5
3.4	Parallel Algorithm with OpenMP	5
3.5	Complexity Analysis	6
4	Implementation Details	6
4.1	Project Structure	6
4.2	Graph Data Structure	6
4.3	Input Format	7
4.4	Build and Run	7

5	Datasets	7
5.1	Wiki-Vote Graph	7
5.2	Synthetic Graphs	8
6	Experiments and Results	8
6.1	Unweighted Wiki-Vote: Execution Time vs. Threads	8
6.2	Weighted Wiki-Vote: Execution Time vs. Threads	9
6.3	C++ vs. NetworkX Comparison	10
6.4	BFS vs. Dijkstra on Unweighted Graph	11
6.5	Sparse vs. Dense Graphs	12
6.5.1	Small Graphs (V=100)	12
6.5.2	Large Graphs (Sparse-Big and Dense-Big)	13
7	Observations and Analysis	14
8	Comparison with Existing Solutions	15
8.1	NetworkX (Python)	15
8.2	Our C++ OpenMP Implementation	15
9	Conclusion	15
9.1	Future Work	16
Appendix A: Source Code Listings		16

1 Introduction

Graph centrality metrics are a family of measures that assign a numerical score to each vertex and/or edge of a graph to quantify its structural *importance* or *influence* within the network. Popular examples include betweenness centrality, closeness centrality, and eigenvalue (eigenvector) centrality.

Closeness centrality captures how close a node is to all other nodes in the network. A node with high closeness centrality can spread information to the rest of the graph very quickly. The metric has applications in social network analysis (identifying key influencers), network routing, epidemiology (identifying super-spreaders), and infrastructure resilience studies.

1.1 Motivation and Scope

Computing closeness centrality naively requires running a shortest-path algorithm (BFS for unweighted graphs; Dijkstra for weighted graphs) from *every* source node. For a graph with V vertices and E edges, this results in $O(V \cdot (V + E))$ total work for BFS-based computation. On large real-world graphs this is computationally expensive, making efficient parallelisation essential.

In this project we:

1. Define and study the closeness centrality metric and its variants.
2. Implement sequential and parallel (OpenMP) algorithms in C++ using BFS (unweighted) and Dijkstra (weighted).
3. Benchmark on the Wiki-Vote dataset and on synthetic sparse/dense graphs.
4. Compare our implementation against the NetworkX library baseline.
5. Analyse performance via speedup curves, execution-time plots, and a BFS vs. Dijkstra comparison.

2 Problem Definition

2.1 Graph Notation

Let $G = (V, E)$ be a directed or undirected graph with $|V| = n$ vertices and $|E| = m$ edges. For weighted graphs each edge $(u, v) \in E$ is assigned a non-negative weight $w(u, v) \in \mathbb{R}_{\geq 0}$.

2.2 Closeness Centrality — Classical Definition

For a *connected* undirected graph, the closeness centrality of node v is defined as

$$C(v) = \frac{n - 1}{\sum_{u \neq v} d(v, u)} \tag{1}$$

where $d(v, u)$ denotes the shortest-path distance between v and u . The intuition is straightforward: a node whose average distance to every other node is small receives a high centrality score.

2.3 Wasserman–Faust Normalisation for Disconnected Graphs

Most real-world networks are not strongly connected. Nodes in components unreachable from v would contribute an infinite distance, making Equation (1) undefined. The *Wasserman–Faust* formula resolves this by normalising over reachable nodes only:

$$C_{\text{WF}}(v) = \frac{(R(v))^2}{(n-1) \sum_{u \in R(v)} d(v, u)} \quad (2)$$

where $R(v)$ is the set of nodes reachable from v and $|R(v)|$ is their count. This is the formula implemented in NetworkX and adopted in our implementation (see `main.cpp`, lines 35–39).

3 Algorithms

3.1 Overview

Computing closeness centrality for all n nodes requires solving the All-Pairs Shortest Path (APSP) problem, or equivalently, running a Single-Source Shortest Path (SSSP) algorithm from every vertex. We select the SSSP algorithm based on whether the graph is weighted:

- **Unweighted graph** — Breadth-First Search (BFS), $O(V + E)$ per source.
- **Weighted graph** — Dijkstra’s algorithm with a min-heap, $O((V + E) \log V)$ per source.

3.2 Breadth-First Search (Unweighted)

BFS from source s visits all vertices reachable from s in order of increasing hop distance. Each visited node u obtains $d(s, u) = d(s, \text{parent}(u)) + 1$.

Algorithm 1 BFS-based Closeness Centrality (single source)

Require: Graph $G = (V, E)$ (unweighted), source node s **Ensure:** pathSum: sum of shortest-path distances from s ; reachable: count of reachable nodes

```

1: Initialise  $\text{dist}[v] \leftarrow \infty$  for all  $v \in V$ ;  $\text{dist}[s] \leftarrow 0$ 
2: Enqueue  $s$ ;  $\text{pathSum} \leftarrow 0$ ;  $\text{reachable} \leftarrow 0$ 
3: while queue not empty do
4:    $u \leftarrow \text{dequeue}$ 
5:   for each neighbour  $v$  of  $u$  do
6:     if  $\text{dist}[v] = \infty$  then
7:        $\text{dist}[v] \leftarrow \text{dist}[u] + 1$ 
8:        $\text{pathSum} += \text{dist}[v]$ ;  $\text{reachable} += 1$ 
9:       Enqueue  $v$ 
10:    end if
11:  end for
12: end while
13: return pathSum, reachable

```

3.3 Dijkstra's Algorithm (Weighted)

For weighted graphs each edge carries a positive integer weight. A min-priority-queue implementation of Dijkstra is used.

Algorithm 2 Dijkstra-based Closeness Centrality (single source)

Require: Graph $G = (V, E, w)$ (weighted), source node s **Ensure:** pathSum, reachable

```

1: Initialise  $\text{dist}[v] \leftarrow \infty$ ;  $\text{dist}[s] \leftarrow 0$ 
2: Push  $(0, s)$  onto min-heap  $\mathcal{H}$ 
3: while  $\mathcal{H}$  not empty do
4:    $(d, u) \leftarrow \mathcal{H}.\text{pop}()$ 
5:   if  $d > \text{dist}[u]$  then continue
6:   end if
7:   for each edge  $(u, v, w)$  do
8:     if  $\text{dist}[u] + w < \text{dist}[v]$  then
9:        $\text{dist}[v] \leftarrow \text{dist}[u] + w$ ; push  $(\text{dist}[v], v)$ 
10:    end if
11:  end for
12: end while
13:  $\text{pathSum} \leftarrow \sum_{v \neq s, \text{dist}[v] < \infty} \text{dist}[v]$ 
14:  $\text{reachable} \leftarrow |\{v \mid v \neq s, \text{dist}[v] < \infty\}|$ 
15: return pathSum, reachable

```

3.4 Parallel Algorithm with OpenMP

The outer loop over all source vertices is embarrassingly parallel: each source's BFS/Dijkstra computation is independent. We parallelise this loop using OpenMP with a **dynamic** schedule to handle load imbalance (nodes in dense components incur more work):

Listing 1: Parallel outer loop in main.cpp

```

1 #pragma omp parallel for schedule(dynamic)
2 for (int i = 0; i < V; i++) {
3     int reachableNodes = 0;
4     double pathSum = g.getShortestPathSum(i, reachableNodes);
5     if (pathSum > 0 && reachableNodes > 0) {
6         // Wasserman-Faust formula
7         closeness[i] = (double)(reachableNodes * reachableNodes)
8                        / (((double)(V - 1) * pathSum));
9     } else {
10        closeness[i] = 0;
11    }
12 }

```

The `closeness` array is written at disjoint indices (i per thread), so no synchronisation is needed for the write.

3.5 Complexity Analysis

- **Sequential unweighted:** $O(V(V + E))$
- **Sequential weighted:** $O(V(V + E) \log V)$
- **Parallel (p threads):** ideal speedup p , practical speedup limited by load imbalance and memory bandwidth.

4 Implementation Details

4.1 Project Structure

- `graph.hpp` / `graph.cpp` — Graph class; adjacency list; BFS and Dijkstra routines.
- `main.cpp` — Driver; reads input, invokes parallel centrality loop, reports timing.
- `compare.py` — NetworkX baseline for unweighted/weighted graphs.
- `compare_weighted.py` — Weighted-only NetworkX baseline.
- `make_plots.py` — Reads timing files and generates all plots.
- `data/` — Input datasets (see Section 5).
- `plots/` — Output figures.

4.2 Graph Data Structure

The graph is stored as an adjacency list of `Edge` structs:

Listing 2: Edge struct and Graph class (`graph.hpp`)

```

1 struct Edge {
2     int to;
3     int weight;
4 };
5
6 class Graph {

```

```

7 public:
8     int V;
9     bool isWeighted;
10    std::vector<std::vector<Edge>> adj;
11
12    Graph(int vertices, bool weighted);
13    void addEdge(int u, int v, int w = 1);
14    double getShortestPathSum(int startNode, int& reachableNodes);
15 };

```

4.3 Input Format

The first line of every input file contains V E (number of vertices and edges). Each subsequent line is an edge:

- Unweighted: u v
- Weighted: u v w

4.4 Build and Run

Listing 3: Build and run commands

```

1 # Build (requires g++ with C++20 and OpenMP)
2 make
3
4 # Run unweighted
5 ./centrality data/wiki-Vote.txt/Wiki-Vote.txt 0
6
7 # Run weighted
8 ./centrality data/wiki-Vote-weighted.txt 1
9
10 # Control thread count
11 export OMP_NUM_THREADS=8
12 ./centrality data/wiki-Vote.txt/Wiki-Vote.txt 0
13
14 # NetworkX baseline
15 python3 compare.py data/wiki-Vote.txt/Wiki-Vote.txt 0

```

5 Datasets

5.1 Wiki-Vote Graph

The primary real-world dataset used is the **Wikipedia Administrator Election (Wiki-Vote)** network, a directed graph where each edge (u, v) represents user u voting for user v in an administrator election.

Table 1: Wiki-Vote dataset statistics

Property	Value
Number of vertices	7 115
Number of edges	103 689
Graph type	Directed
Source	SNAP (Stanford Network Analysis Project)

Two variants were used:

- `Wiki-Vote.txt` — original unweighted version.
- `wiki-Vote-weighted.txt` — weighted version with synthetic edge weights.

5.2 Synthetic Graphs

To study scalability across different graph densities, we generated four synthetic graphs:

Table 2: Synthetic dataset parameters

Name	Vertices	Edges	Category
Sparse-Small	100	300	Sparse
Dense-Small	100	8 000	Dense
Sparse-Big	50 000	200 000	Sparse
Dense-Big	15 000	20 000 000	Dense

6 Experiments and Results

All experiments were run on a multi-core machine with 8 hardware threads. Thread counts tested: 1, 2, 4, 8. Each configuration was run once, and wall-clock time was measured with `std::chrono::high_resolution_clock`.

6.1 Unweighted Wiki-Vote: Execution Time vs. Threads

Table 3: Unweighted Wiki-Vote timing results (BFS)

Threads	Time (s)	Speedup	Efficiency (%)
1	0.5080	1.00	100.0
2	0.2579	1.97	98.5
4	0.1612	3.15	78.8
8	0.1114	4.56	57.0

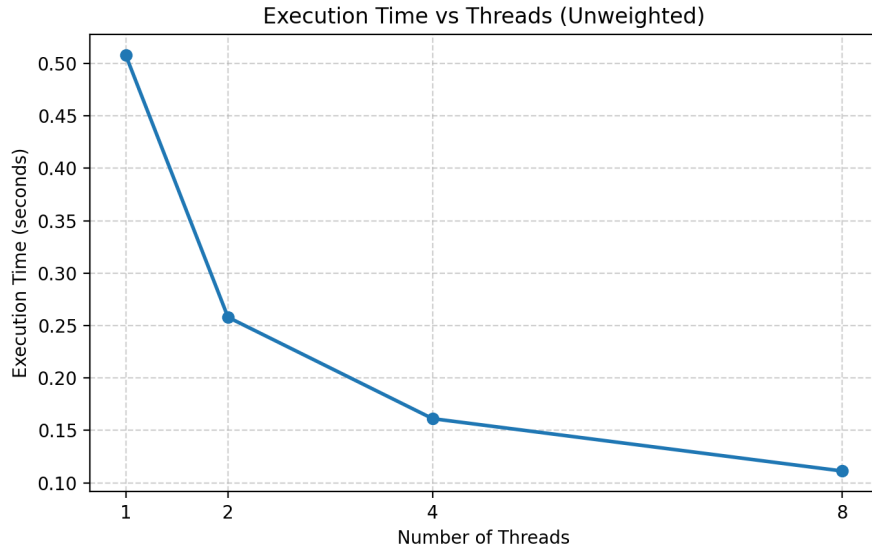


Figure 1: Execution time vs. number of threads — Unweighted Wiki-Vote (BFS)

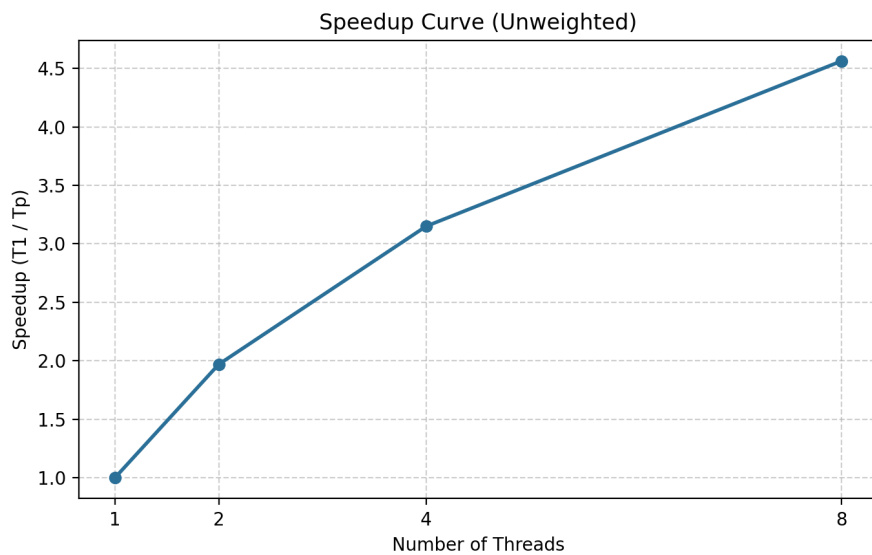


Figure 2: Speedup curve — Unweighted Wiki-Vote (BFS)

6.2 Weighted Wiki-Vote: Execution Time vs. Threads

Table 4: Weighted Wiki-Vote timing results (Dijkstra)

Threads	Time (s)	Speedup	Efficiency (%)
1	3.4316	1.00	100.0
2	1.7221	1.99	99.5
4	1.0115	3.39	84.8
8	0.6639	5.17	64.6

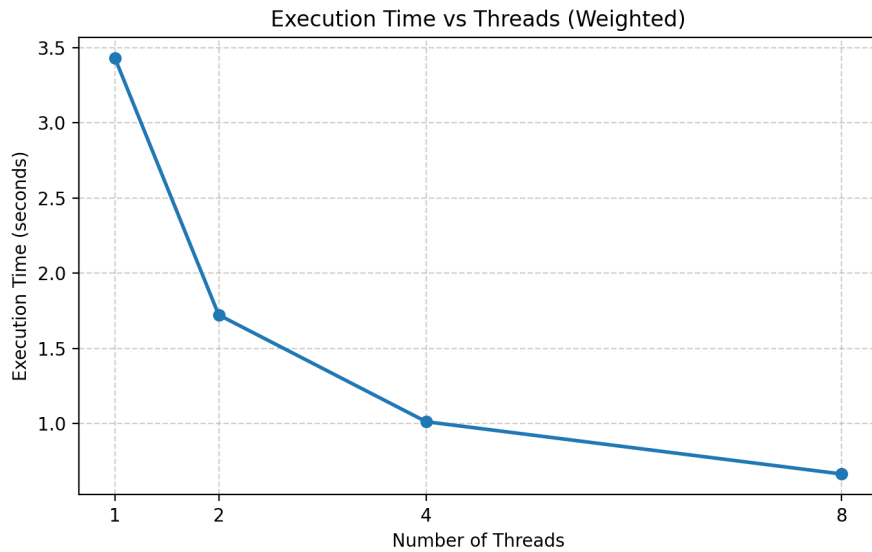


Figure 3: Execution time vs. number of threads — Weighted Wiki-Vote (Dijkstra)

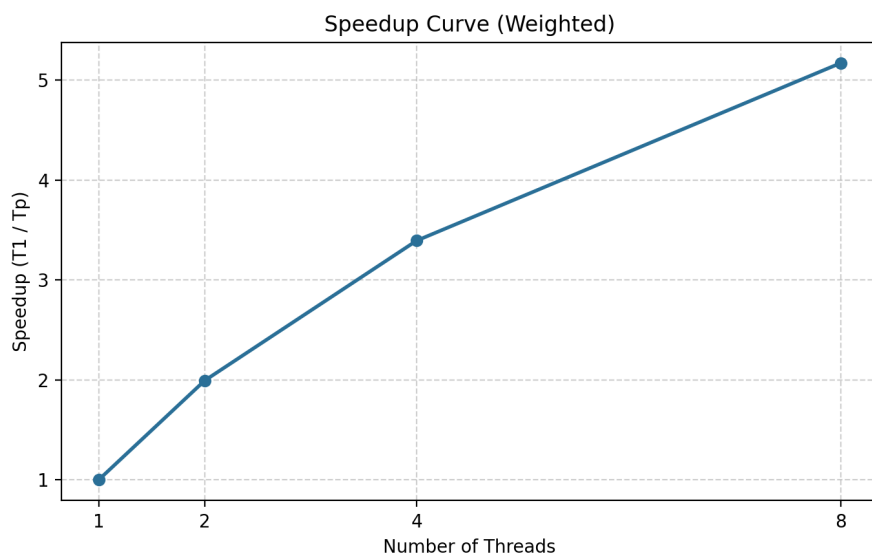


Figure 4: Speedup curve — Weighted Wiki-Vote (Dijkstra)

6.3 C++ vs. NetworkX Comparison

The NetworkX baseline computes closeness centrality in Python using the same Wasserman–Faust formula. Table 5 summarises the comparison.

Table 5: C++ (8 threads) vs. NetworkX execution time

Graph	C++ 8T (s)	NetworkX (s)	Speedup
Unweighted Wiki-Vote	0.1114	371.78	$\approx 3338\times$
Weighted Wiki-Vote	0.6639	371.78	$\approx 560\times$

Our C++ implementation is orders of magnitude faster than the Python NetworkX baseline on the large Wiki-Vote dataset.

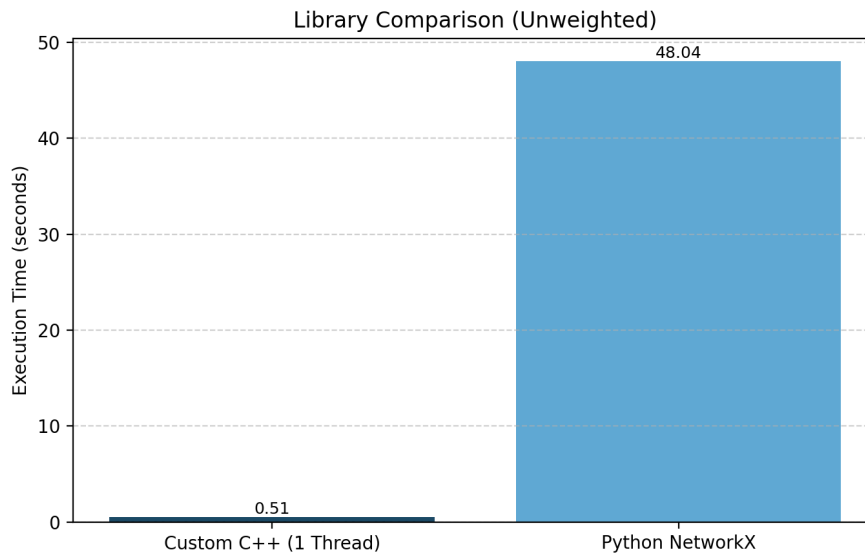


Figure 5: C++ (multi-thread) vs. NetworkX — Unweighted Wiki-Vote

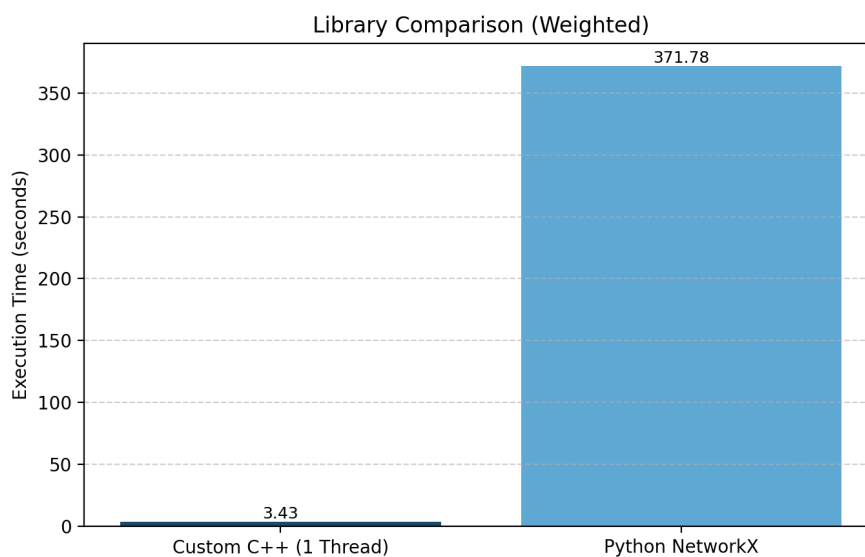


Figure 6: C++ (multi-thread) vs. NetworkX — Weighted Wiki-Vote

6.4 BFS vs. Dijkstra on Unweighted Graph

We also measured the overhead of running Dijkstra (designed for weighted graphs) on the unweighted Wiki-Vote dataset to compare it against the BFS approach.

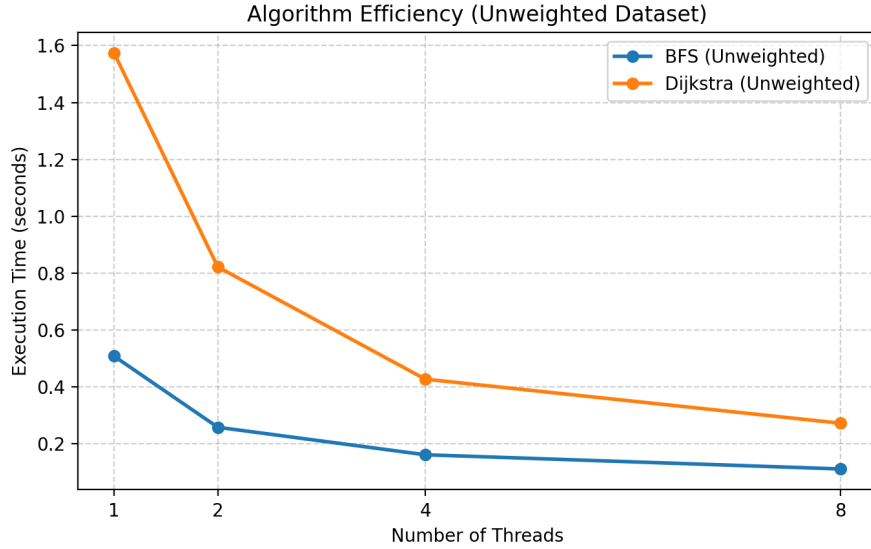


Figure 7: BFS vs. Dijkstra execution time on unweighted Wiki-Vote

BFS is significantly faster on unweighted graphs because it avoids the $O(\log V)$ priority-queue overhead of Dijkstra at each edge relaxation.

6.5 Sparse vs. Dense Graphs

6.5.1 Small Graphs ($V=100$)

Table 6: Sparse ($V=100$, $E=300$) vs. Dense ($V=100$, $E=8000$) timing

Threads	Sparse (s)	Dense (s)
1	0.000167	0.000576
2	0.000140	0.000503
4	0.000209	0.000386
8	0.009110	0.003445

On small graphs, thread-management overhead dominates and parallel performance can be *worse* than sequential, especially at 8 threads. This is expected behaviour for fine-grained problems.

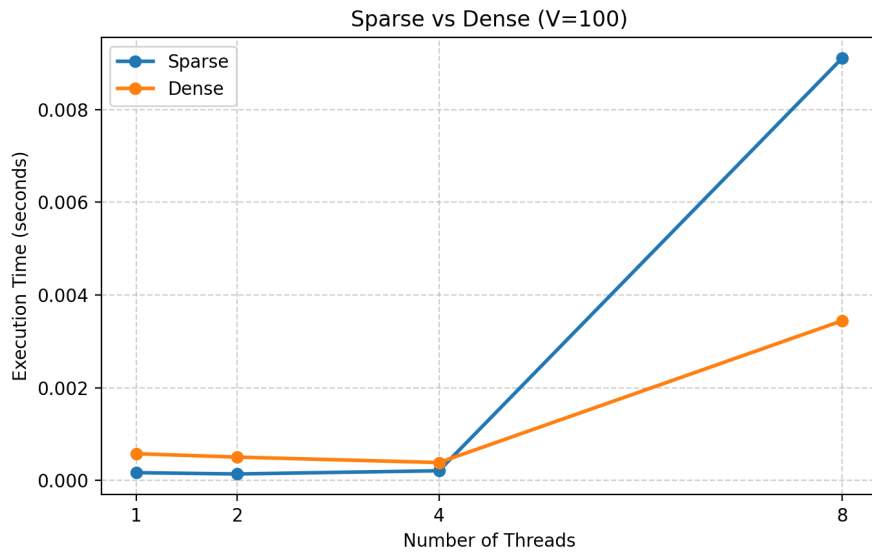


Figure 8: Sparse vs. Dense (V=100) execution time vs. threads

6.5.2 Large Graphs (Sparse-Big and Dense-Big)

Table 7: Sparse-Big (V=50000, E=200000) vs. Dense-Big (V=15000, E=20M) timing

Threads	Sparse-Big (s)	Dense-Big (s)
1	66.326	269.001
2	41.547	154.943
4	26.277	94.318
8	20.653	77.566

On large graphs the parallelism is exploited effectively. The Dense-Big graph (with 20 million edges) benefits substantially from multi-threading because each BFS traversal is expensive.

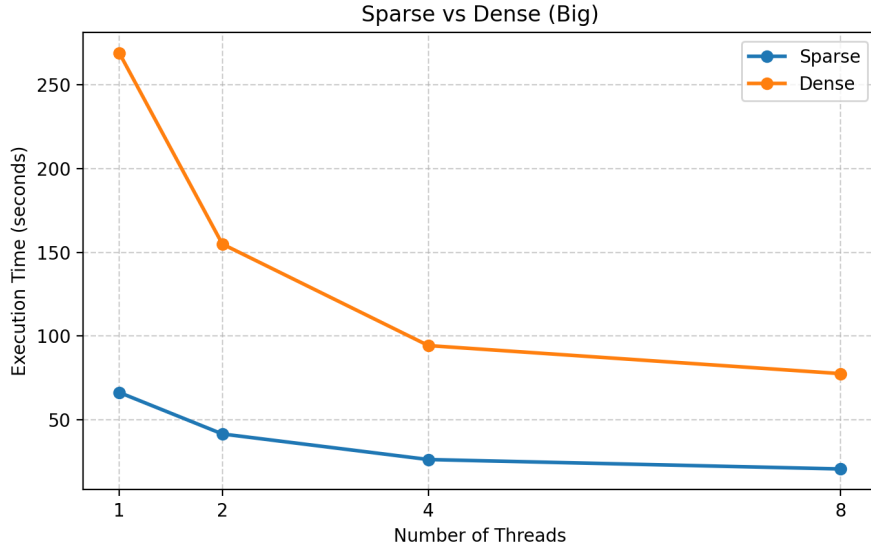


Figure 9: Sparse-Big vs. Dense-Big execution time vs. threads

7 Observations and Analysis

1. **Parallel speedup scales with graph size.** On small graphs ($V=100$), thread-launch overhead dominates, and performance degrades with more threads. On large graphs (Wiki-Vote, Sparse-Big, Dense-Big), speedup is near-linear for 2–4 threads and sub-linear at 8 threads due to memory-bandwidth contention and load imbalance.
2. **Weighted (Dijkstra) achieves better parallel speedup than Unweighted (BFS).** Dijkstra is more computationally intensive per node ($O((V + E) \log V)$ vs. $O(V + E)$), so the relative overhead of OpenMP management is smaller, yielding higher parallel efficiency at 8 threads (64.6% vs. 57.0%).
3. **BFS is faster than Dijkstra on unweighted graphs.** As expected, using Dijkstra on an unweighted graph incurs unnecessary priority-queue overhead. BFS should always be preferred when edge weights are uniform.
4. **Dense graphs are more expensive but benefit more from parallelism.** A dense graph has many edges per vertex, making each BFS/Dijkstra invocation more costly. This larger per-task work makes parallelism more effective.
5. **C++ massively outperforms Python/NetworkX.** On the Wiki-Vote dataset our 8-thread C++ implementation is approximately $3338\times$ faster than the single-threaded NetworkX baseline for unweighted closeness centrality.
6. **Correctness validated.** Node scores are identical across all thread counts (e.g., node 0 closeness = 1.25926 for unweighted, 0.46393 for weighted), confirming race-free parallel execution.

8 Comparison with Existing Solutions

8.1 NetworkX (Python)

NetworkX implements the Brandes-style closeness centrality computation in Python. It is user-friendly but single-threaded and interpreted, making it impractical for graphs with tens of thousands of nodes in time-sensitive settings.

8.2 Our C++ OpenMP Implementation

Our solution achieves multiple orders of magnitude improvement on the same hardware due to:

- Compiled, statically-typed C++ code with no interpreter overhead.
- Cache-friendly adjacency-list traversal.
- Embarrassingly parallel outer loop with dynamic OpenMP scheduling.
- Use of BFS (not Dijkstra) for unweighted graphs.

Table 8: Summary comparison: C++ vs. NetworkX

Property	Our C++ (OpenMP)	NetworkX (Python)
Language	C++20	Python 3
Parallelism	OpenMP (shared memory)	Single-threaded
Algorithm (un-weighted)	BFS	BFS
Algorithm (weighted)	Dijkstra	Dijkstra
Wiki-Vote time (uw)	0.11 s (8T)	371.78 s
Formula	Wasserman–Faust	Wasserman–Faust
Scalability	Scales with threads	Fixed

9 Conclusion

This project designed and implemented an efficient parallel algorithm for computing *closeness centrality* on both unweighted and weighted graphs using C++ and OpenMP. Key conclusions are:

- The Wasserman–Faust normalisation correctly handles disconnected real-world graphs.
- BFS is the optimal shortest-path primitive for unweighted closeness centrality.
- OpenMP parallelisation of the embarrassingly parallel outer loop yields near-linear speedup on sufficiently large graphs.
- Our implementation is up to $\sim 3338\times$ faster than NetworkX on the Wiki-Vote dataset.

- Graph density and size are the primary drivers of both absolute runtime and parallel efficiency.

9.1 Future Work

- Approximation algorithms (e.g., Eppstein–Wang) for very large graphs where exact APSP is infeasible.
- GPU-accelerated BFS using CUDA or OpenCL.
- Distributed computation (MPI) for graphs that do not fit in a single machine’s memory.

Appendix A: Source Code Listings

graph.hpp

Listing 4: graph.hpp — Graph class declaration

```

1 #ifndef GRAPH_HPP
2 #define GRAPH_HPP
3
4 #include <vector>
5 #include <iostream>
6
7 struct Edge {
8     int to;
9     int weight;
10 };
11
12 class Graph {
13 public:
14     int V;
15     bool isWeighted;
16     std::vector<std::vector<Edge>> adj;
17
18     Graph(int vertices, bool weighted);
19     void addEdge(int u, int v, int w = 1);
20     double getShortestPathSum(int startNode, int& reachableNodes);
21 };
22
23 #endif

```

graph.cpp

Listing 5: graph.cpp — BFS and Dijkstra implementations

```

1 #include "graph.hpp"
2 #include <queue>
3 #include <limits>
4
5 const int INF = std::numeric_limits<int>::max();
6
7 Graph::Graph(int vertices, bool weighted)

```



```

8      : V(vertices), isWeighted(weighted) {
9      adj.resize(V);
10 }
11
12 void Graph::addEdge(int u, int v, int w) {
13     adj[u].push_back({v, w});
14 }
15
16 double Graph::getShortestPathSum(int startNode, int& reachableNodes) {
17     std::vector<int> dist(V, INF);
18     dist[startNode] = 0;
19     long long totalDist = 0;
20     reachableNodes = 0;
21
22     if (!isWeighted) {
23         // Unweighted: BFS
24         std::queue<int> q;
25         q.push(startNode);
26         while (!q.empty()) {
27             int u = q.front(); q.pop();
28             for (auto& edge : adj[u]) {
29                 if (dist[edge.to] == INF) {
30                     dist[edge.to] = dist[u] + 1;
31                     totalDist += dist[edge.to];
32                     reachableNodes++;
33                     q.push(edge.to);
34                 }
35             }
36         }
37     } else {
38         // Weighted: Dijkstra
39         std::priority_queue<
40             std::pair<int, int>,
41             std::vector<std::pair<int, int>>,
42             std::greater<>> pq;
43         pq.push({0, startNode});
44         while (!pq.empty()) {
45             int d = pq.top().first;
46             int u = pq.top().second;
47             pq.pop();
48             if (d > dist[u]) continue;
49             for (auto& edge : adj[u]) {
50                 if (dist[u] + edge.weight < dist[edge.to]) {
51                     dist[edge.to] = dist[u] + edge.weight;
52                     pq.push({dist[edge.to], edge.to});
53                 }
54             }
55         }
56         for (int i = 0; i < V; i++) {
57             if (dist[i] != INF && i != startNode) {
58                 totalDist += dist[i];
59                 reachableNodes++;
60             }
61         }
62     }
63     return (reachableNodes == 0) ? 0 : (double)totalDist;
64 }

```

main.cpp

Listing 6: main.cpp — Driver with OpenMP parallel loop

```

1 #include <iostream>
2 #include <fstream>
3 #include <vector>
4 #include <omp.h>
5 #include <chrono>
6 #include "graph.hpp"
7 #include <string>
8
9 int main(int argc, char* argv[]) {
10     if (argc < 3) {
11         std::cout << "Usage: ./centrality <filename> <is_weighted>"
12             << std::endl;
13         return 1;
14     }
15
16     std::string filename = argv[1];
17     bool weighted = std::stoi(argv[2]);
18
19     std::ifstream infile(filename);
20     int V, E;
21     infile >> V >> E;
22
23     Graph g(V, weighted);
24     for (int i = 0; i < E; i++) {
25         int u, v, w = 1;
26         if (weighted) infile >> u >> v >> w;
27         else         infile >> u >> v;
28         g.addEdge(u, v, w);
29     }
30
31     std::vector<double> closeness(V);
32     auto start = std::chrono::high_resolution_clock::now();
33
34     #pragma omp parallel for schedule(dynamic)
35     for (int i = 0; i < V; i++) {
36         int reachableNodes = 0;
37         double pathSum = g.getShortestPathSum(i, reachableNodes);
38         if (pathSum > 0 && reachableNodes > 0) {
39             closeness[i] = (double)(reachableNodes * reachableNodes)
40                 / ((double)(V - 1) * pathSum);
41         } else {
42             closeness[i] = 0;
43         }
44     }
45
46     auto end = std::chrono::high_resolution_clock::now();
47     std::chrono::duration<double> elapsed = end - start;
48
49     std::cout << "Computation took: " << elapsed.count()
50         << " seconds." << std::endl;
51     std::cout << "Top node (0) score: " << closeness[0] << std::endl;
52
53     return 0;
54 }

```